

AI Audit and What I Learned

EECE 5155 – Wireless Sensor Networks and IoT Systems

Aiisha Matsungo — NUID: 002530298 — March 2026

Task 1 – Extend the Grafana Dashboard (30%)

What I prompted

“I have a Grafana dashboard from the seminar with 2 panels (time series and gauge). What two panels should I add to reach 4 total, and what Flux queries would I use? Carefully read the Professor’s instructions!”

What AI got right

It correctly identified that the two seminar panels were already built, so suggestions complemented rather than duplicated existing work.

What AI got wrong / what I fixed

The initial queries used `range(start: -30d)` which returned no data because the dataset is from 2018, not the last 30 days. I changed all queries to hardcoded absolute ranges (2018-10-01T00:00:00Z to 2018-12-31T00:00:00Z). The gauge also displayed entirely red because Threshold mode defaulted to **Percentage** instead of **Absolute** – at 28 and 35 on a 15–55 scale, percentage mode placed the yellow boundary at $\approx 26^\circ\text{C}$ and the red boundary at $\approx 29^\circ\text{C}$, collapsing the yellow band and making 31°C appear red. Switching Threshold mode to Absolute fixed the colour mapping immediately.

What I learned

Grafana’s `range(start: -30d)` is relative to *now*, so historical datasets return empty unless absolute timestamps are hardcoded or the dashboard time picker is set to the data’s actual date range. Gauge threshold mode matters as much as the threshold values: the same numbers produce completely different colour bands in Percentage vs Absolute mode. Choosing the right visualisation type is also as important as writing the correct query – the mean-by-location result reads as a bare number in Table view but becomes immediately comparable as a Bar chart.

Task 2 – Data Quality Report (40%)

What I prompted

Uploaded the seminar slides PDF, then the repo file listing screenshot.

What AI got right

Once the slides were uploaded, Claude correctly identified all four required checks ($\mu \pm 3\sigma$ outliers per device, duplicate timestamps, time gaps, stuck sensor) and the expected table output format from slide 24. It also correctly identified that the script should read `IOT-temp.csv` directly with pandas.

What AI got wrong / what I fixed

Claude rewrote the script from scratch twice unnecessarily. The first version used `influxdb_client` to query InfluxDB (wrong – the assignment reads from CSV). The second used pandas but with its own structure. Both were redundant because the professor had already provided a complete working `data_quality.py` in the course repo (ebaenamar/ECE5155_practical_iot_cloud). The professor's version handles the unusual CSV column names (`room_id/id`, `out/in`) with a conditional check, uses `format="mixed"` for robust timestamp parsing, and computes z-scores using `groupby().transform()` (vectorised, not a loop). The correct first step was simply to run:

```
python data_quality.py --csv IOT-temp.csv
```

A second script (`task2_report.py`) was written to produce the exact summary table format shown on slide 24, filling in the ? values the rubric expects.

Actual output (IOT-temp.csv)

Device		Readings	Mean	Outliers	Duplicates	Max Gap	
Stuck?							

Room	Admin_Out	77,261	36.3	0	0	11d	13h
	No						
Room	Admin_In	20,345	30.5	30	78,253	11d	14h
	No						

What I learned

AI will confidently generate code from scratch even when a reference implementation already exists – it has no visibility into a repo unless you share it. The correct workflow is to check the professor's starter code *before* prompting AI. I also learned how `groupby().transform()` works in pandas: it applies a function per group but returns a Series aligned to the original DataFrame index, so the z-score column can be added back directly without a merge. The 78,253 duplicate figure (80% of all rows) was the most significant finding: multiple device IDs logging to the same room+timestamp collapse into the same `device` key when grouped by room and location, causing massive timestamp collisions. The 30 outliers in Room Admin_In at 21–41 °C are physically plausible – they are flagged because that sensor's standard deviation is only 2.2 °C, making the 3σ band very narrow (± 6.6 °C), not because the readings are genuinely erroneous.

Task 3 – Pipeline Design for Final Project (30%)

What I prompted

“Design the pipeline for my final project” – initially without uploading the PES. After the first output, PES v1 was uploaded, followed by the seminar slides.

What AI got right

The layered pipeline structure (sensing → transport → storage → visualisation) was correct, and the MQTT topic hierarchy, JSON payload format, and Grafana panel suggestions were technically sound for a generic IoT stack.

What AI got wrong / what I fixed

The professor provided the seminar pipeline (MQTT → Mosquitto → InfluxDB → Grafana) as the in-class stack. Task 3 asked us to design the pipeline for *our own final project*, which is a different thing. Claude defaulted to the seminar stack instead of reading my PES, which specifies HTTP POST directly to ThingSpeak with a custom HTML/JS dashboard on an **ESP8266**. Claude assumes everything, one has to explain and interact with AI so as to get the actual sense.

What I learned

The same underlying pipeline pattern – Sensor → MCU → Network → Cloud DB → Dashboard – applies whether you use MQTT + InfluxDB or HTTP POST + ThingSpeak. The implementation differs based on scale and constraints, not the concept. Both stacks move data from a sensor to a screen; the tools are interchangeable, the concept is not. Knowing this means you can swap any layer without redesigning the whole system. My project uses HTTP POST instead of MQTT because my constraints (2/3 sensors, \$50 budget, timeline, ESP8266 on reliable campus WiFi) made pub/sub unnecessary overhead. MQTT adds value when you have many sensors publishing at once, intermittent connectivity where guaranteed delivery matters, or QoS requirements for critical data — none of which apply to a 2-node quiet zone monitor sending every 30 seconds on eduroam. For this MVP, HTTP POST goes directly from sensor to ThingSpeak in one step; MQTT would add a broker, a Python bridge, and InfluxDB — three extra failure points with no reliability gain. This maps directly to Constraint #1 (cost) and Constraint #3 (timeline) in my PES: implementation simplicity was more valuable than protocol sophistication.